WEEK 3 · DAY 1 · OVERVIEW

Dated Data and Walk-Forward Validation



Created by **Jeff Lenamon**

ABOUT 55 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Every corporate spread on the desk is quoted over a Treasury yield.

A desk quant asks a question about the curve's shape: given the 2 Yr and the 10 Yr on a date, what 30 Yr goes with them on that same date?

For ten days the course scored bonds and card accounts, rows with no order; today's rows are dates.

All three yields sit on one row, so the model's output is a fitted 30 Yr yield given the same day's 2 Yr and 10 Yr.

It describes how the long end sat against them inside the file's span, and a gap between actual and fitted is the model's miss, not a view on rates.

OBJECTIVES

Score a regression tree and a straight line with shuffled folds, then again in time order.

Find out why the two scores differ, fit ridge and lasso where the tenors are near-copies, catch lookahead hiding in a feature, and test whether the misses run.

ABOUT 55 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

Your Route Through Day 1

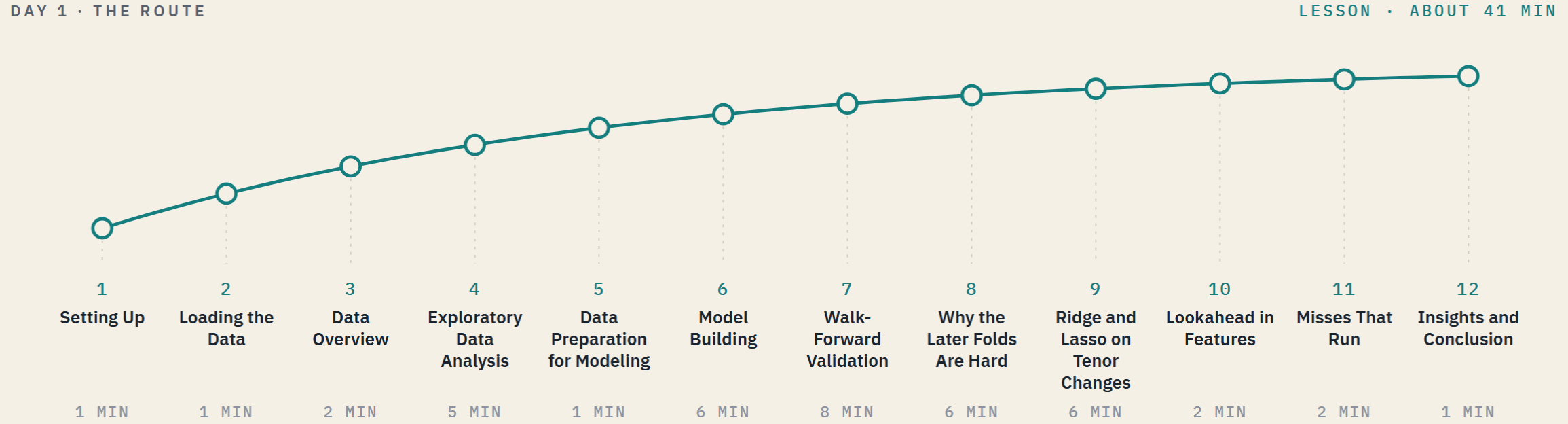


FIGURE 1 Twelve sections, in order, with about how long each takes.

Time. About 41 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SEED

```
import os
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.tree import DecisionTreeRegressor
from sklearn.pipeline import make_pipeline
from statsmodels.stats.stattools import durbin_watson
```

Run this cell and move on: it fixes the seed and finds the course data.

Read the Treasury file, one row per date.

Loading the Data

THE FILE

```
yields = pd.read_csv(DATA + "treasury_par_yields.csv", parse_dates=["date"], index_col="date")
```

THE COLUMNS

- **`date`**: the market date, 2015-01-02 to 2026-10-02
- **`1 Mo`, `3 Mo`, `6 Mo`, `1 Yr`, `2 Yr`, `3 Yr`, `5 Yr`, `7 Yr`, `10 Yr`, `20 Yr`, `30 Yr`**: par yields, percent
- **`1.5 Month`, `2 Mo`, `4 Mo`**: tenors that start partway through the file, not used

`parse_dates=["date"]` reads the date column as dates, and `index_col="date"` makes it the row labels, as in Course 1's time series notebook.

Data Overview

SHAPE AND DATES

```
(2940, 14)
From 2015-01-02 to 2026-10-02
Each date once: True  In order: True
```

COLUMNS THAT START LATE

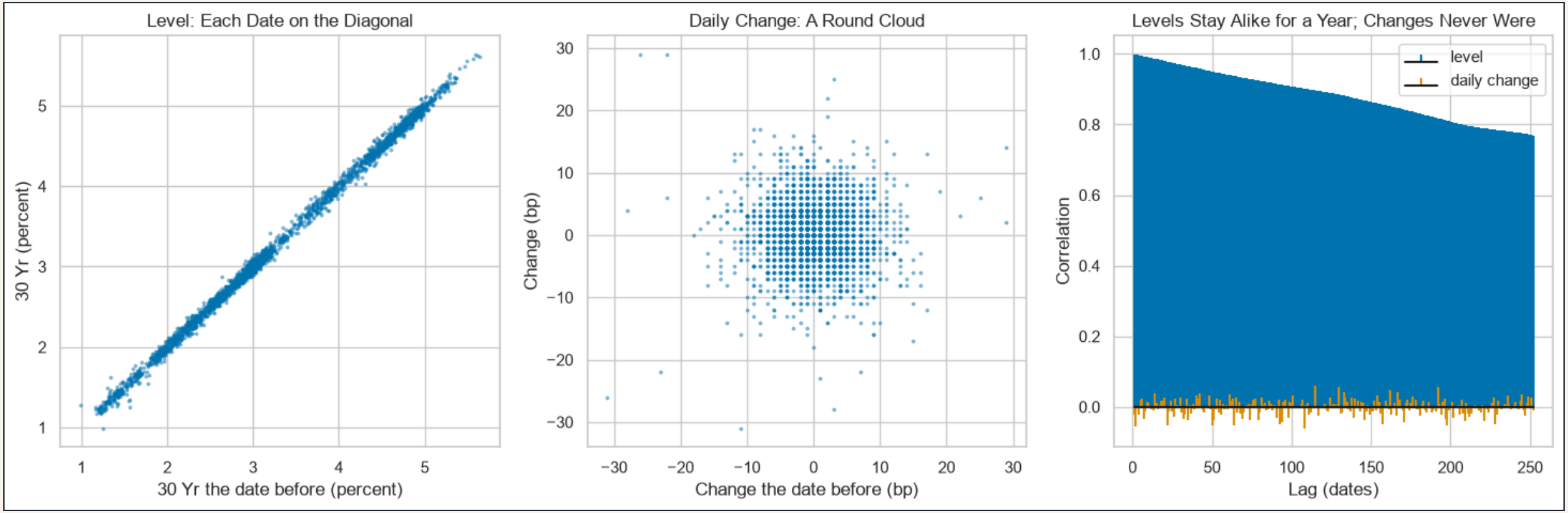
```
1.5 Month    2532
2 Mo          949
4 Mo         1951
dtype: int64
2 Mo starts on 2018-10-16
4 Mo starts on 2022-10-19
1.5 Month starts on 2025-02-18
```

MEAN, LOWEST AND HIGHEST YIELD, PERCENT

	MEAN	MIN	MAX
1 Mo	2.10	0.00	6.02
3 Mo	2.16	0.00	5.63
6 Mo	2.22	0.02	5.61
1 Yr	2.24	0.04	5.49
2 Yr	2.28	0.09	5.19
3 Yr	2.34	0.10	5.03
5 Yr	2.47	0.19	5.09
7 Yr	2.64	0.36	5.19
10 Yr	2.76	0.52	5.29
20 Yr	3.13	0.87	5.68
30 Yr	3.22	0.99	5.64

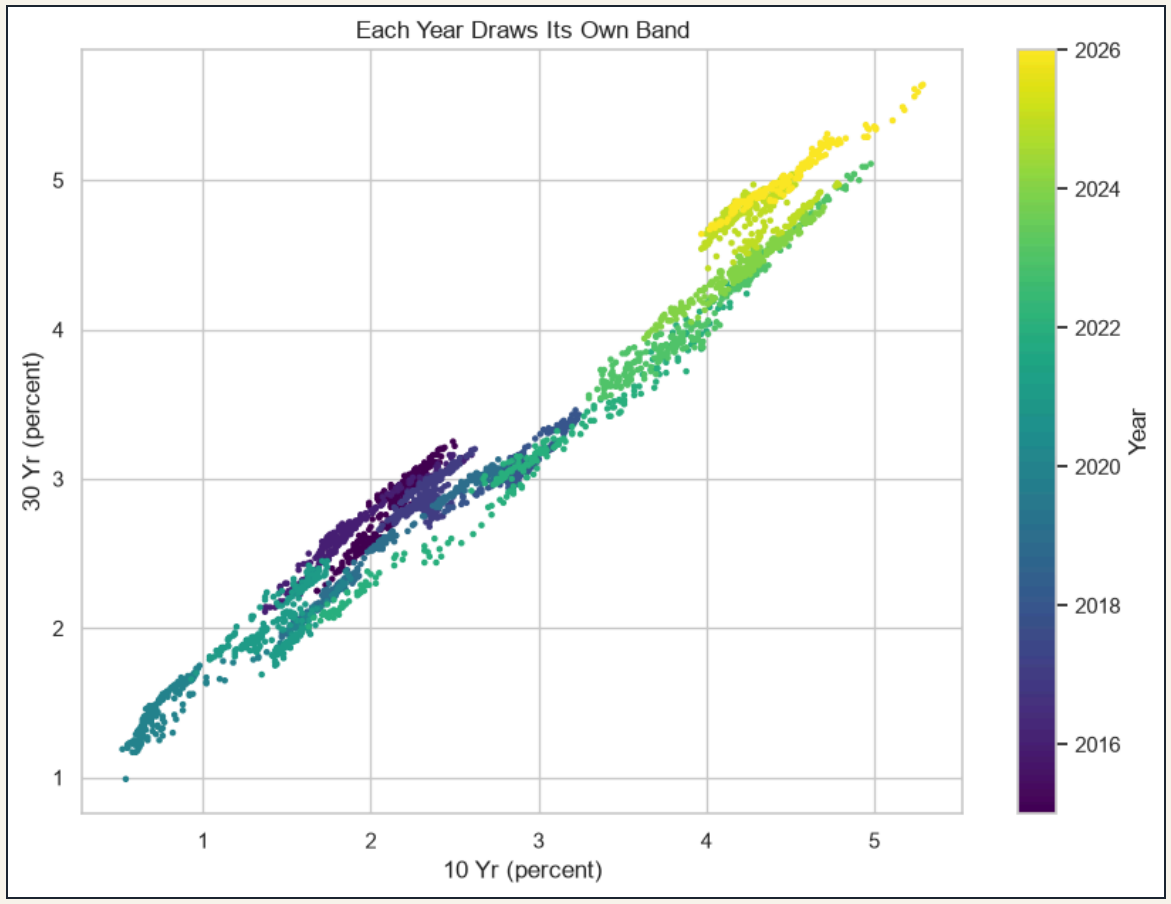
Eleven tenors are complete and the dates are in order, so we keep those eleven and look at what makes dated rows different.

Exploratory Data Analysis



A date's level is a near-copy of the dates around it; its daily change is not. Any split that shuffles dates puts a near-copy of a test date among the training rows.

Data Preparation for Modeling



FEATURES AND TARGET

```
X = curve[["2 Yr", "10 Yr"]]
y = curve["30 Yr"]
```

WHERE THE YEAR BANDS COME FROM

The 30 Yr sat 70 bp over the 10 Yr on average in 2015, 14 bp in 2023, and 54 bp in 2026.

NO SPLIT HERE

No split is made here: the folds are the splits.

Every number comes from one row, so the model describes a date's curve shape and reads nothing later.

Model Building

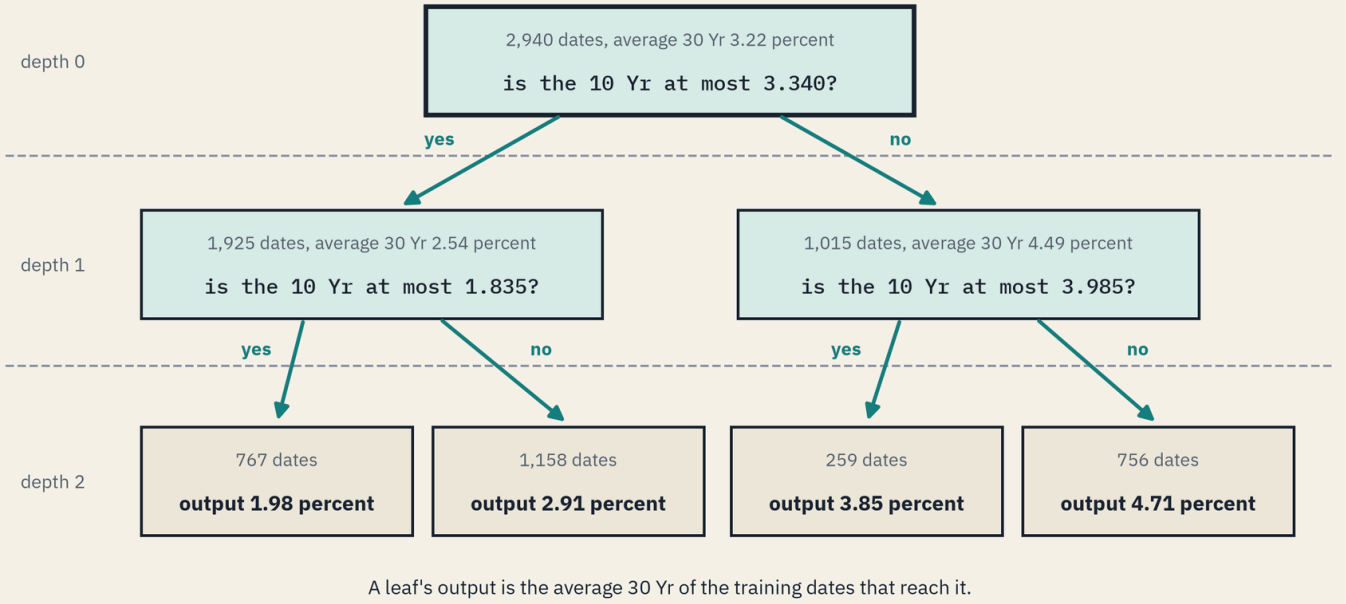


FIGURE 2 A tree two questions deep on all 2,940 dates, each leaf the average 30 Yr of its dates.

THE TWO MODELS

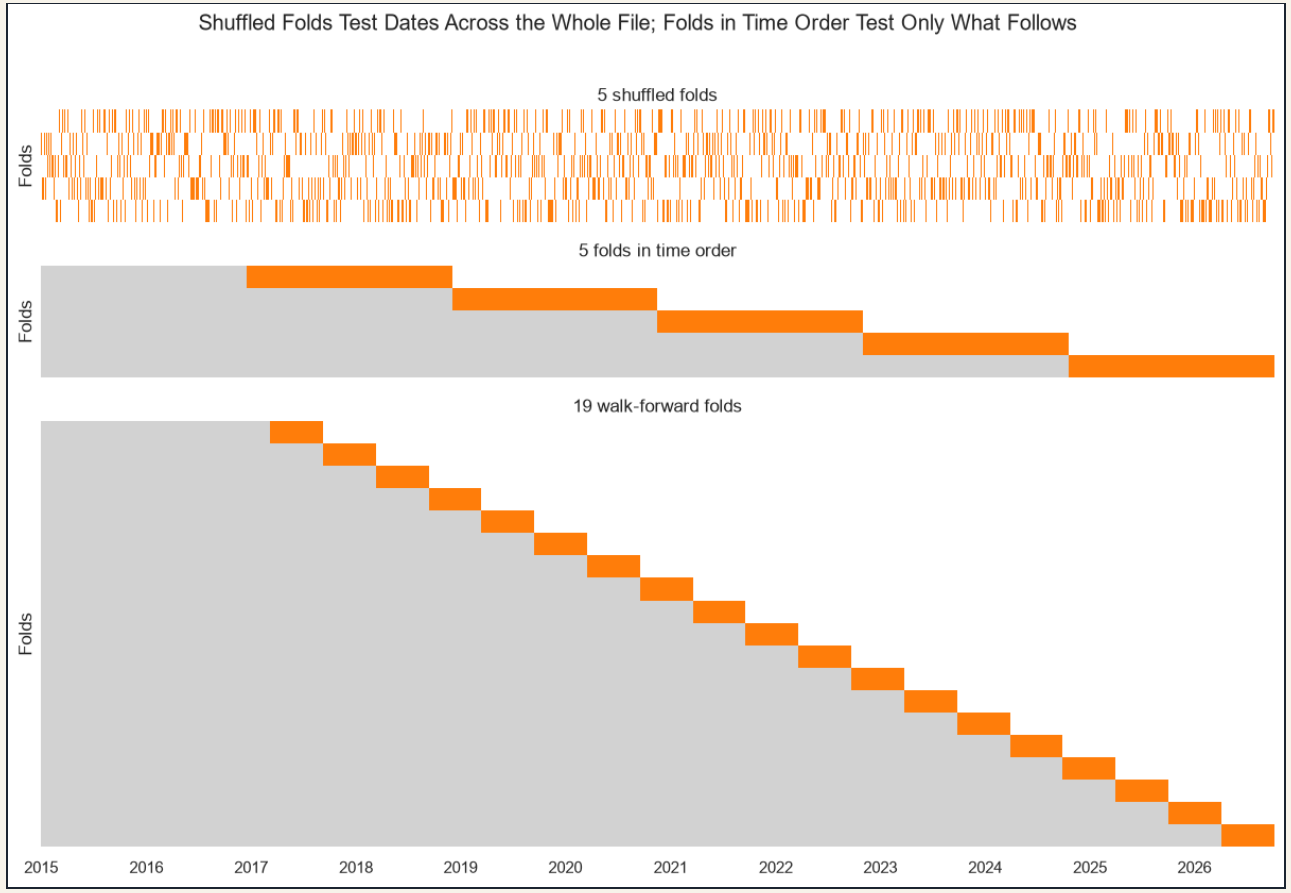
```
tree = DecisionTreeRegressor(random_state=SEED)
line = LinearRegression()
```

ROWS FROM A TEST DATE TO ITS NEAREST TRAINING DATE

```
1    571
2     17
Name: count, dtype: int64
```

571 of 588 test dates have a training date one row away, and none is more than two away.

Walk-Forward Validation



NINETEEN FOLDS

```
wf = TimeSeriesSplit(n_splits=19, test_size=126)
splits = list(wf.split(X))
```

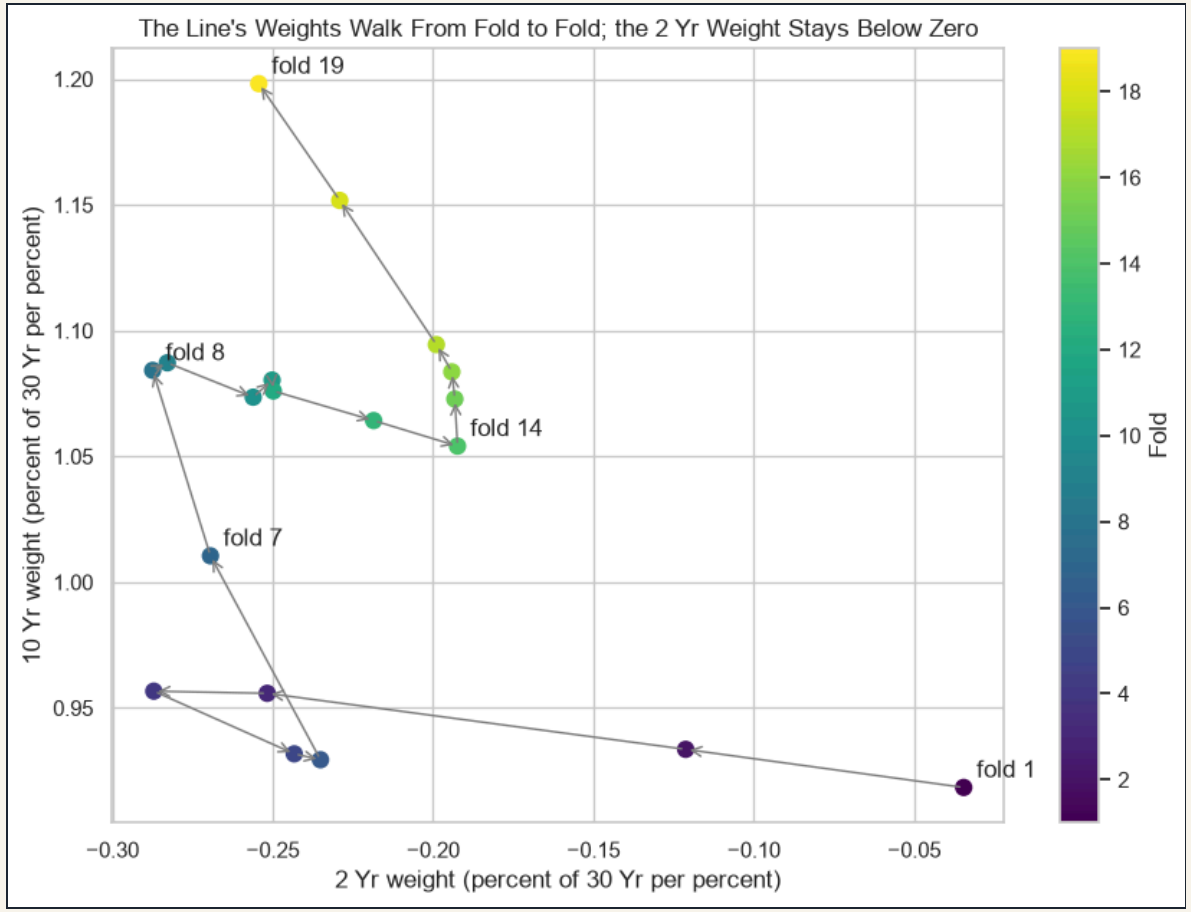
HOW IT WORKS

Walk-forward validation fits each fold on a training window and tests it on the window of dates right after it.

An **expanding** window always starts on the first date; a **rolling** window keeps one length and slides.

In time order each lane is grey up to a point, orange right after it, and white beyond: nothing later is fitted.

Why the Later Folds Are Hard



MISSES IN THE LINE'S WORST FOLDS

FOLD	LINE MEAN MISS	LINE MAE	FEATURES OUTSIDE
10	-24.3	24.3	0
17	26.8	26.8	0
18	26.4	26.4	0

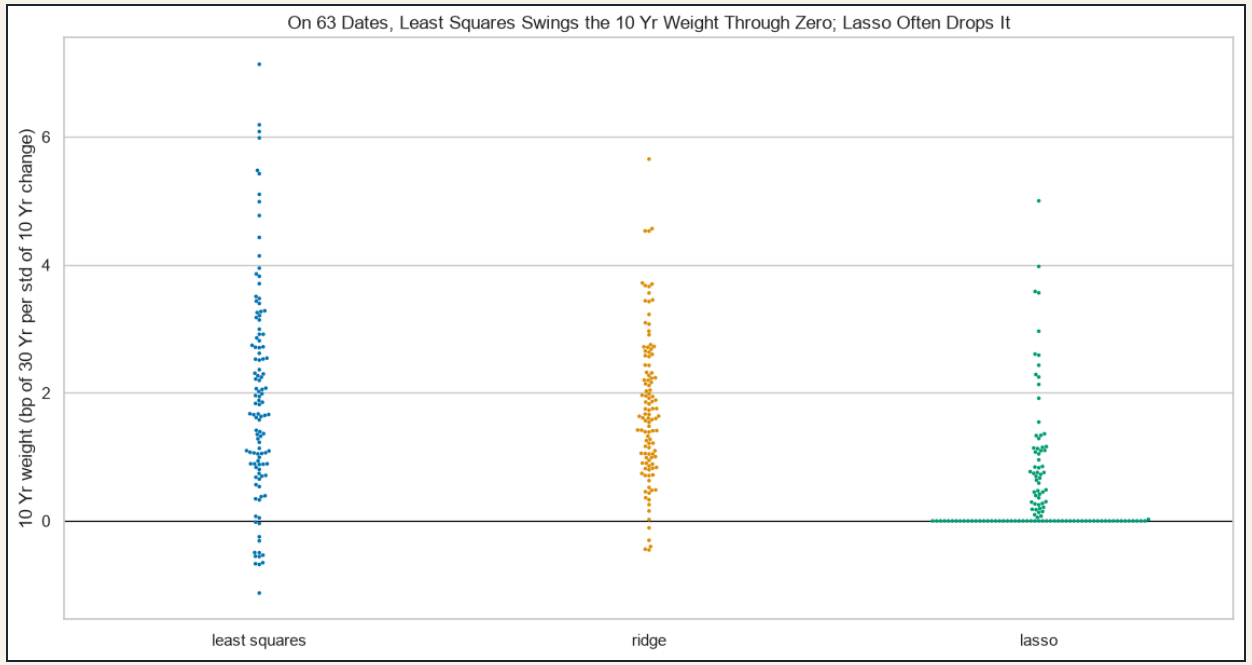
THE LINE'S WEIGHTS

The 2 Yr weight is below zero in all 19 folds: holding the 10 Yr fixed, a higher 2 Yr goes with a lower fitted 30 Yr (Day 3's words).

The 10 Yr weight runs from 0.918 to 1.198 and steps up between folds 7 and 8, from 1.010 to 1.084.

The line's mean miss is as large as its MAE: every miss sits on one side, below the line's value in fold 10 and above it in folds 17 and 18.

Ridge and Lasso on Tenor Changes



THE SPREAD OF THE WEIGHTS

On 63 dates the 10 Yr weight's spread falls from 1.65 (least squares) to 1.14 (ridge) and 0.95 (lasso).

TEN NEAR-COPIES: THE VIFS

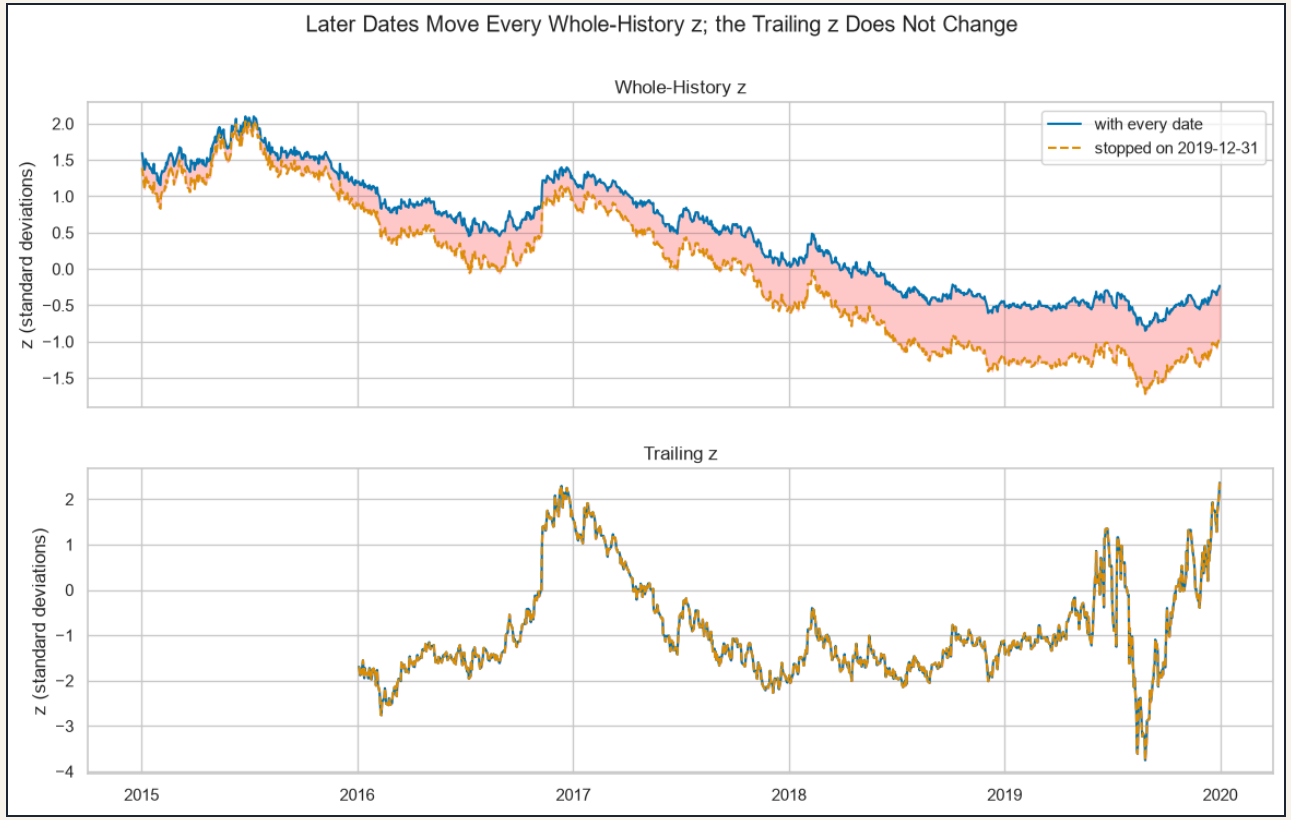
```
7 Yr     60.58
10 Yr    52.65
5 Yr     45.63
3 Yr     23.48
20 Yr    16.57
```

WINDOWS OF 63 DATES

	R-SQUARED	WEIGHT STD	AT ZERO
least squares	0.9401	1.6500	0
ridge	0.9414	1.1373	0
lasso	0.9423	0.9519	55

Least squares puts it below zero in 13 windows; lasso never does, and drops the 10 Yr altogether in 55.

Lookahead in Features



EACH Z-SCORE ON 2019-12-31

```
Whole-history z: with every date -0.2303  stopped -0.9311
Trailing z:      with every date 2.3634  stopped 2.3634
Whole-history values up to the cut that change: 1250 of 1250
```

THE CUT TEST

Course 3's no-lookahead test, the cut test: stop the data on a cut date, recompute, and compare.

Walk-forward keeps later dates out of training; a feature built from the whole history carries them back in.

Read Durbin-Watson on level and change misses.

Misses That Run

THE STATISTIC

The **Durbin-Watson** statistic sums the squared step from each miss to the next and divides by the sum of the squared misses.

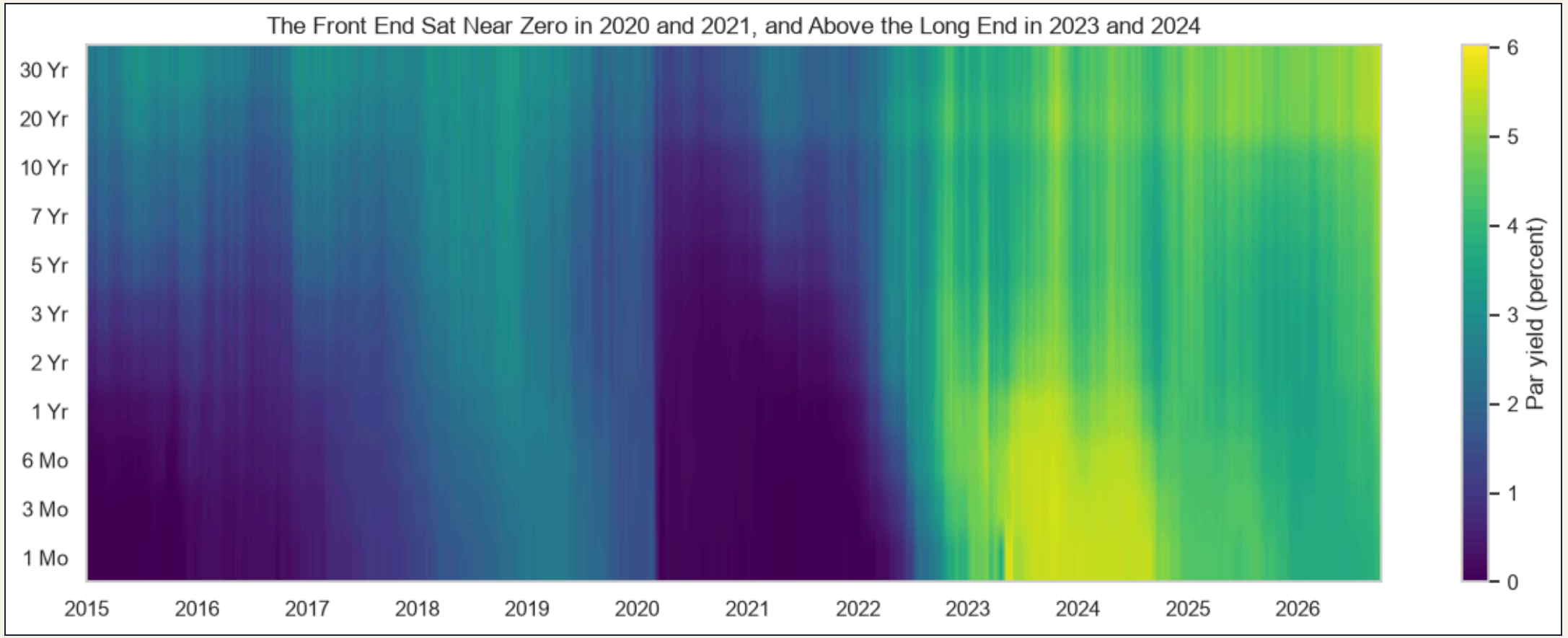
It is about 2 when a miss is unrelated to the one before, near 0 when each miss copies the last, and near 4 when each miss takes the other sign from the last.

THE LINE'S MISSES ON LEVELS

```
print("Durbin-Watson:", round(durbin_watson(level_misses), 3))
print("Sign changes:", (level_misses * level_misses.shift() < 0).sum(), "in", len(level_misses), "dates")
```

```
Durbin-Watson: 0.011
Sign changes: 27 in 2394 dates
```

Insights and Conclusion



Everything here describes this file inside its span, 2015 to 2026.

You Can Now

- Read lag plots and a correlogram, and say why a shuffled split flatters a model on dates.
- Score a model walk-forward with **`TimeSeriesSplit`** and **`cross_validate`**, and read the spread across folds.
- Say what a regression tree does with a date outside its training range.
- Fit ridge and lasso with **`make_pipeline`** on expanding and rolling windows.
- Find lookahead with the cut test, and read a Durbin-Watson statistic.

NEXT
WEEK 3 · DAY 2

Scaling and K-Means

On Day 12 the target goes away: k-means groups bonds by their numbers alone.